

20 Recursion - Zero to Hero questions list

July 18, 2026

1 Recursion in C — Practice Questions (50 Questions with Hints Only)

The questions are arranged from **Beginner** → **Intermediate** → **Advanced**.

2 Beginner Level (1–15)

2.0.1 1. Write a recursive function to print numbers from 1 to N.

Hint: Print after the recursive call reaches the base case or before it, depending on the desired order.

2.0.2 2. Write a recursive function to print numbers from N to 1.

Hint: Print before making the recursive call.

2.0.3 3. Write a recursive function to calculate the factorial of a number.

Hint: Base case: $0! = 1$ or $1! = 1$.

2.0.4 4. Write a recursive function to calculate the sum of first N natural numbers.

Hint: $\text{sum}(n) = n + \text{sum}(n-1)$.

2.0.5 5. Write a recursive function to calculate the power of a number.

Hint: $\text{power}(a, b) = a \times \text{power}(a, b-1)$.

2.0.6 6. Write a recursive function to calculate the cube of a number.

Hint: Use recursion or reuse the power function.

2.0.7 7. Write a recursive function to print even numbers from 2 to N.

Hint: Increase by 2 in each recursive call.

2.0.8 8. Write a recursive function to print odd numbers from 1 to N.

Hint: Skip even numbers by increasing by 2.

2.0.9 9. Write a recursive function to count down from N to 0.

Hint: Stop when the value becomes 0.

2.0.10 10. Write a recursive function to calculate the product of two numbers using repeated addition.

Hint: Add one number repeatedly until the multiplier becomes 0.

2.0.11 11. Write a recursive function to find the square of a number using repeated addition.

Hint: Square = $n + \text{square}(n-1)$ pattern or repeated addition.

2.0.12 12. Write a recursive function to calculate the sum of digits of a number.

Hint: Extract the last digit using $\% 10$.

2.0.13 13. Write a recursive function to count the number of digits in an integer.

Hint: Divide by 10 in each recursive call.

2.0.14 14. Write a recursive function to reverse a number.

Hint: Process one digit at a time while maintaining place value.

2.0.15 15. Write a recursive function to check whether a number is even or odd.

Hint: Reduce the number recursively until it reaches 0 or 1.

3 Intermediate Level (16–35)

3.0.1 16. Write a recursive function to generate the Fibonacci sequence.

Hint: $F(n) = F(n-1) + F(n-2)$.

3.0.2 17. Write a recursive function to find the Nth Fibonacci number.

Hint: Use two recursive calls.

3.0.3 18. Write a recursive function to calculate the Greatest Common Divisor (GCD).

Hint: Apply Euclid's Algorithm recursively.

3.0.4 19. Write a recursive function to calculate the Least Common Multiple (LCM).

Hint: Use the recursive GCD function.

3.0.5 20. Write a recursive function to check whether a number is prime.

Hint: Test divisibility using a recursive helper function.

3.0.6 21. Write a recursive function to check whether a number is a palindrome.

Hint: Reverse the number recursively or compare digits recursively.

3.0.7 22. Write a recursive function to check whether a number is an Armstrong number.

Hint: Recursively process each digit.

3.0.8 23. Write a recursive function to calculate the sum of an array.

Hint: Sum one element and recurse on the remaining array.

3.0.9 24. Write a recursive function to find the largest element in an array.

Hint: Compare one element with the recursive result.

3.0.10 25. Write a recursive function to find the smallest element in an array.

Hint: Reduce the problem by one element in each call.

3.0.11 26. Write a recursive function to search an element in an array.

Hint: Compare the current element and recurse.

3.0.12 27. Write a recursive function to count even numbers in an array.

Hint: Check one element per recursive call.

3.0.13 28. Write a recursive function to count odd numbers in an array.

Hint: Similar to counting even numbers.

3.0.14 29. Write a recursive function to reverse an array.

Hint: Swap first and last elements recursively.

3.0.15 30. Write a recursive function to calculate the average of array elements.

Hint: First recursively compute the sum.

3.0.16 31. Write a recursive function to find the length of a string.

Hint: Stop when '\0' is reached.

3.0.17 32. Write a recursive function to reverse a string.

Hint: Swap characters from both ends recursively.

3.0.18 33. Write a recursive function to count vowels in a string.

Hint: Process one character at a time.

3.0.19 34. Write a recursive function to copy one string into another.

Hint: Copy one character before the recursive call.

3.0.20 35. Write a recursive function to compare two strings.

Hint: Compare one character at a time.

4 Advanced Level (36–50)

4.0.1 36. Write a recursive function to convert a decimal number to binary.

Hint: Divide by 2 until the quotient becomes 0.

4.0.2 37. Write a recursive function to convert a decimal number to octal.

Hint: Divide by 8 recursively.

4.0.3 38. Write a recursive function to convert a decimal number to hexadecimal.

Hint: Divide by 16 and process the remainder.

4.0.4 39. Write a recursive implementation of Binary Search.

Hint: Divide the search interval into halves.

4.0.5 40. Write a recursive implementation of Linear Search.

Hint: Check one element in each recursive call.

4.0.6 41. Write a recursive Bubble Sort.

Hint: One pass followed by recursion on the reduced array.

4.0.7 42. Write a recursive Selection Sort.

Hint: Find the minimum recursively.

4.0.8 43. Write a recursive Insertion Sort.

Hint: Sort the first $n-1$ elements recursively, then insert the last.

4.0.9 44. Write a recursive Merge Sort.

Hint: Divide the array into halves and merge recursively.

4.0.10 45. Write a recursive Quick Sort.

Hint: Partition the array and recursively sort both partitions.

4.0.11 46. Solve the Tower of Hanoi problem recursively.

Hint: Move $n-1$ disks, move the largest disk, then move $n-1$ disks again.

4.0.12 47. Generate all permutations of a string recursively.

Hint: Swap characters and recurse for the remaining positions.

4.0.13 48. Generate all subsets (power set) of a given set recursively.

Hint: Include or exclude each element.

4.0.14 49. Solve the N-Queens problem using recursion (Backtracking).

Hint: Place one queen per row and backtrack when conflicts occur.

4.0.15 50. Solve a Sudoku puzzle using recursion (Backtracking).

Hint: Fill one empty cell at a time, recursively trying valid numbers and backtracking when necessary.